

ACADEMIC RESEARCH CASE STUDY • EMPIRICAL CODE STYLOMETRICS

Zero-Shot LLM Code Synthesis vs. Production-Hardened Human Reference Code: A Stylometric and Structural Case Study

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Study Scope: 136 Complete Programs (10 Pre-AI Reference Libraries vs 126 LLM Generations)

Abstract

The rapid adoption of Large Language Models (LLMs) for automated code synthesis has sparked interest in evaluating the structural and qualitative properties of machine-generated code. However, existing benchmarks often focus strictly on functional pass rates (e.g., HumanEval, MBPP) rather than stylometric, performance, and maintenance characteristics. This paper presents an empirical case study comparing **136 complete code implementations**, contrasting zero-shot synthetic generations from three frontier LLM architectures (*Google Gemini 3.5 Flash*, *OpenAI GPT-5.6 Sol*, and *Anthropic Claude Sonnet 4.6*) against a reference baseline of 10 production-hardened standard library functions authored by prominent human engineers prior to the LLM era (2017–2018 code from React 16, Go 1.10, Redis 5.0, Linux Kernel 4.14, Rust stdlib, PyTorch 1.0, and FastHTTP).

Our quantitative analysis reveals statistically significant stylometric divergence: synthetic implementations exhibit **+174% lines of code (LOC) expansion** (Mean = 41.17 ± 31.36 LOC vs. 15.00 ± 6.43 LOC human, Mann-Whitney U = 305.5, $p = 0.0069$), elevated comment density ($9.23\% \pm 11.15\%$ synthetic vs. $1.43\% \pm 4.29\%$ human, $p = 0.0152$), and higher vertical whitespace ratios ($13.54\% \pm 7.60\%$ synthetic vs. $5.54\% \pm 6.59\%$ human, $p = 0.0027$). We catalogue empirical occurrence rates for seven structural patterns—including micro-helper function fragmentation (**54.8%** of synthetic programs), trivial syntax-echo comments (**22.2%**), and contextual invariant omissions (**4.8%**). Finally, we explicitly document threats to validity, noting that comparing zero-shot LLM output against battle-tested open-source reference code highlights the gap between initial machine generation and production-hardened software rather than an intrinsic limit of synthetic intelligence.

1. Introduction

Automated code generation powered by Large Language Models (LLMs) has transitioned from snippet autocompletion to end-to-end function synthesis. While LLMs achieve high pass rates on standard coding benchmarks, code quality in production software depends heavily on non-functional dimensions: maintainability, memory alignment, cache locality, control flow clarity, and domain-specific state invariant enforcement.

2. Related Work

Recent research has increasingly examined the quality, verbosity, and correctness of LLM-generated code: Binkley et al. (2023) and Jesse et al. (2023) observed significant verbosity in LLM code; Kabir et al. (2023) analyzed tutorial explanation habits in LLM outputs; Ugare et al. (2024) systematically categorized performance bugs in LLM code; and Nguyen et al. (2023) documented security and domain invariant omissions in Copilot outputs.

3. Methodology & Dataset Composition

Pre-AI Human Baseline Reference Dataset (n=10): Extracted from 2017–2018 repositories: React 16 Fiber Scheduler (Andrew Clark / Dan Abramov), React 16 shallowEqual, Go 1.10 strings.Builder (Russ Cox / Brad Fitzpatrick), Rust `slice::rotate_left`, PyTorch 1.0 `clamp_out`, Redis 5.0 `raxInsert` (antirez), TypeScript 3.0

isIdentifierStart (Anders Hejlsberg), Linux Kernel 4.14 eBPF htab_map_lookup_elem, FastHTTP caseInsensitiveCompare, and Rust Vec::retain.

Sampling Protocol & Prompting: Single-shot stateless generation (temperature = 0.0 / default minimum variance), max_tokens = 750-800. System prompt: "Write clean, production-quality code. Output only the code, no explanation."

Dataset Breakdown (N=136 Total): Part A (Research Tasks) = 76 completed runs; Part B (Human Benchmark Recreations) = 14 runs + 50 baseline evaluation versions + 10 Human reference files = 136 records total.

4. Quantitative Stylometric Results & Statistical Significance

Stylometric Metric	Human Reference (n=10)	Synthetic Models (N=126)	Mann-Whitney U	p-value	Statistical Significance
Lines of Code (LOC)	15.00 ± 6.43 [10.4, 19.6]	41.17 ± 31.36 [35.6, 46.7]	305.5	p = 0.0069	Statistically Significant (p < 0.01)
Comment Density (%)	1.43% ± 4.29% [0.0, 4.5]	9.23% ± 11.15% [7.3, 11.2]	359.0	p = 0.0152	Statistically Significant (p < 0.05)
Explicit Type Annotations	1.50 ± 1.28 [0.6, 2.4]	8.56 ± 10.70 [6.7, 10.4]	337.0	p = 0.0142	Statistically Significant (p < 0.05)
Helper Method Count	0.20 ± 0.60 [0.0, 0.6]	0.63 ± 1.56 [0.4, 0.9]	550.0	p = 0.3604	Not Significant (p > 0.05)
Return Statement Count	1.70 ± 1.42 [0.7, 2.7]	2.56 ± 2.16 [2.2, 2.9]	478.0	p = 0.1983	Not Significant (p > 0.05)
Vertical Whitespace (%)	5.54% ± 6.59% [0.8, 10.3]	13.54% ± 7.60% [12.2, 14.9]	271.0	p = 0.0027	Statistically Significant (p < 0.01)

5. Empirical Frequency Analysis of 7 Structural Patterns

Pattern Identifier & Description	Observed Frequency (k / 126)	Percentage (%)
Pattern 1: Structural Micro-Fragmentation — Decomposing simple tasks into ≥ 2 helper functions or extra class wrappers.	69 / 126	54.8%
Pattern 2: Contextual Invariant Omission — Omitting domain safety checks (struct copy guards, null prototype handling, Object.is).	6 / 126	4.8%
Pattern 3: Trivial Syntax-Echo Comments — Writing comments that directly parrot line syntax (# Increment counter).	28 / 126	22.2%
Pattern 4: Functional Iterator Closures in Hot Loops — Using .every(), .map(), .forEach() in performance hot loops.	5 / 126	4.0%
Pattern 5: In-Loop Mutating Array Shifts — Regressing runtime complexity from $O(N)$ to $O(N^2)$ via vector removals in loops.	1 / 126	0.8%
Pattern 6: Asynchronous State & Timer Lifecycle Leaks — Omitting clearTimeout() or mutating subscriber lists live.	Qualitative Sub-sample	—
Pattern 7: Compiler Vectorization Obstacles — Using nested std::min(std::max(...)) calls that hinder SIMD auto-vectorization.	1 / 126	0.8%

6. Model Stylometric Profiles

Google Gemini 3.5 Flash: Hyper-pedagogical profile, highest comment density (49.48%), extensive JSDoc wrappers.

OpenAI GPT-5.6 Sol: Enterprise minimal profile, lowest comment density (0.88%), coarse-grained global mutex locks.

Anthropic Claude Sonnet 4.6: Comprehensive test-inclusive profile, highest line count (73.5 LOC avg), embedded unit tests.

7. Threats to Validity

Construct Validity: Comparing zero-shot LLM output against production-hardened open-source code authored by senior engineers measures the gap between initial machine generation and hardened software, not an average human developer.

Internal Validity: LLMs were evaluated strictly in single-shot mode. Multi-turn developer interaction, compiler error feedback, and code review shrink this quality gap.

Statistical Validity: While $N=126$ synthetic generations provide high statistical power, the human reference sample ($n=10$) is small. Non-parametric Mann-Whitney U tests were applied to address sample size asymmetry.

8. Conclusion & References

This study establishes statistically significant stylometric differences between zero-shot LLM code and production-hardened human reference code, providing empirical baseline metrics and frequency counts for automated code synthesis research.

References: Binkley et al. (2023) TSE; Jesse et al. (2023) ISSTA; Kabir et al. (2023) EMSE; Nguyen et al. (2023) ICSE; Ugare et al. (2024) PACMPL.